Week 2 Structured Notes: Memory and Dynamic Arrays

Topic Overview

Week 2 explains how arrays are stored in memory and how programmers can create arrays dynamically while a program is running.

In Week 1, the focus was on using arrays: declaring them, initializing them, accessing elements, traversing them, finding sum, maximum, minimum, and printing memory addresses.

Week 2 moves one level deeper. It explains where arrays live in memory and how to manage memory manually using pointers.

This week mainly covers:

1. Stack memory
2. Heap memory
3. Stack arrays
4. Heap arrays
5. Pointers
6. `malloc` in C
7. `free` in C
8. `new[]` in C++
9. `delete[]` in C++
10. Memory leaks
11. Copying heap arrays
12. Differences between stack arrays and heap arrays
13. Common mistakes in dynamic memory
14. Full practice programs

---

1. What You Should Know Before Week 2

Before studying Week 2, you should already understand these Week 1 ideas:

- A variable stores one value.
- An array stores many values under one name.
- Array indexing starts from 0 in C and C++.
- A 5-element array has indexes 0 to 4.
- A loop is used to visit every array element.
- Array elements are stored side by side in memory.
- Each array element has a memory address.

- Accessing one array element is fast because the computer can calculate its location directly.

Example:

```
int A[5] = {2, 4, 6, 8, 10};
```

Logical view:

```
Index:  0   1   2   3   4
Value:  2   4   6   8   10
```

If one integer uses 4 bytes, the memory may look like this:

```
A[0] address = 200
A[1] address = 204
A[2] address = 208
A[3] address = 212
A[4] address = 216
```

This side-by-side storage is called contiguous memory.

---

2. Main Idea of Week 2

Arrays can be stored in two main memory areas:

```
1. Stack memory
2. Heap memory
```

Simple meaning:

```
Stack = automatic memory
Heap  = manually controlled memory
```

A normal array like this is usually a stack array:

```
int A[5];
```

A dynamic array like this is a heap array:

```c
int *P = (int*)malloc(5 * sizeof(int));
```

In C++:

```cpp
int *P = new int[5];
```

The important difference is this:

```
Stack arrays are created automatically.
Heap arrays are created manually using pointers.
```

---

3. Stack Memory

Meaning of Stack Memory

Stack memory is a memory area used for local variables inside functions.

Example:

```c
int main() {
    int x = 10;
    int A[5] = {1, 2, 3, 4, 5};
    return 0;
}
```

Here, `x` and `A` are local variables inside `main()`. They are commonly stored in stack memory.

Simple Explanation

Imagine stack memory as a temporary workspace.

When a function starts, memory is given to its local variables.

When the function ends, that memory is automatically removed.

So, for stack memory:

```
Created automatically
Removed automatically
Fast to use
Usually limited in size
```

---

4. Stack Arrays

Meaning of a Stack Array

A stack array is a normal fixed-size array created inside a function.

Example:

```c
int A[5] = {2, 4, 6, 8, 10};
```

This creates 5 integer spaces.

```
A[0] = 2
A[1] = 4
A[2] = 6
A[3] = 8
A[4] = 10
```

Code Example

```c
#include <stdio.h>

int main() {
    int A[5] = {2, 4, 6, 8, 10};

    for (int i = 0; i < 5; i++) {
        printf("%d ", A[i]);
    }

    return 0;
}
```

Output:

```
2 4 6 8 10
```

Important Point

A stack array has a fixed size.

If you create this:

```
int A[5];
```

You cannot directly stretch it into this:

```
int A[10];
```

That does not work because the size of a normal array is fixed after creation.

Beginner Example

Imagine you reserve 5 seats in a row:

```
[ ] [ ] [ ] [ ] [ ]
```

Later, you want 10 seats.

You cannot just stretch the same row because the space next to it may already be used by someone else.

Arrays have the same problem. They need continuous memory. If the memory after the array is already used, the array cannot grow directly.

---

5. Heap Memory

Meaning of Heap Memory

Heap memory is used when a program needs memory while it is running.

This is called dynamic memory allocation.

Dynamic means the size can be decided during runtime.

Example situation:

```
The user enters how many marks they want to store.
```

The program does not know the array size before running.

So instead of using a fixed array, we can create a heap array.

Simple Explanation

Heap memory is like requesting extra space when you need it.

But there is a rule:

```
If you request heap memory, you must release it when you finish using it.
```

In C:

```
malloc creates heap memory.
free releases heap memory.
```

In C++:

```
new[] creates a heap array.
delete[] releases a heap array.
```

---

6. Heap Arrays

Meaning of a Heap Array

A heap array is an array created in heap memory.

In C:

```c
int *P = (int*)malloc(5 * sizeof(int));
```

In C++:

```
int *P = new int[5];
```

Here, `P` is a pointer.

The actual array is created in the heap.

The pointer stores the starting address of that heap array.

Simple diagram:

```
Stack memory:
P stores the address of the heap array

Heap memory:
[ ] [ ] [ ] [ ] [ ]
```

After storing values:

```
Stack memory:
P → address of heap block

Heap memory:
[3] [5] [7] [9] [11]
```

---

7. Pointers

Meaning of a Pointer

A pointer is a variable that stores a memory address.

Normal variable:

```
int x = 10;
```

This stores a value.

Pointer variable:

```
int *P;
```

This stores an address.

Simple View

```
x = 10
```

This means `x` stores the value 10.

```
P = address of another memory location
```

This means `P` stores where something is located in memory.

Why Heap Arrays Need Pointers

When an array is created in heap memory, the program needs a way to remember where that array starts.

That is the job of the pointer.

Example:

```
int *P = (int*)malloc(5 * sizeof(int));
```

`P` points to the first element of the heap array.

So:

```
P[0] = first element
P[1] = second element
P[2] = third element
P[3] = fourth element
P[4] = fifth element
```

Even though `P` is a pointer, we can use it like an array.

---

8. Creating a Heap Array in C Using malloc

Basic Syntax

```
int *P = (int*)malloc(5 * sizeof(int));
```

Breakdown:

```
int *P              → creates an integer pointer
malloc              → requests memory from heap
5                   → number of elements
sizeof(int)         → size of one integer
5 * sizeof(int)     → total memory needed
```

If one integer takes 4 bytes:

```
5 * 4 = 20 bytes
```

So this line requests memory for 5 integers.

Full C Example

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *P = (int*)malloc(5 * sizeof(int));

    P[0] = 3;
    P[1] = 5;
    P[2] = 7;
    P[3] = 9;
    P[4] = 11;

    for (int i = 0; i < 5; i++) {
        printf("%d ", P[i]);
    }

    free(P);
```

```
    return 0;
}
```

Output:

```
3 5 7 9 11
```

Important Header File

To use `malloc` and `free`, include this header:

```
#include <stdlib.h>
```

Without this, the program may show errors or warnings.

---

9. Why We Use sizeof(int)

You may think this is enough:

```
malloc(20);
```

But that is not a good habit.

Better:

```
malloc(5 * sizeof(int));
```

Reason:

The size of `int` may not be the same on every system. Usually it is 4 bytes, but good programmers do not hardcode that number.

`sizeof(int)` asks the compiler:

```
How many bytes does one int need on this system?
```

So this line is safer:

```c
int *P = (int*)malloc(5 * sizeof(int));
```

---

10. Checking Whether malloc Worked

Sometimes, `malloc` may fail.

This can happen if the system does not have enough memory.

When `malloc` fails, it returns `NULL`.

Safe code:

```c
int *P = (int*)malloc(5 * sizeof(int));

if (P == NULL) {
    printf("Memory allocation failed.");
    return 1;
}
```

This is better than blindly using `P`.

If `P` is `NULL` and you try to use `P[0]`, the program can crash.

---

11. Releasing Heap Memory in C Using free

When you create heap memory using `malloc`, you must release it using `free`.

Example:

```c
free(P);
```

This tells the system:

```
I finished using this memory. You can take it back.
```

Full Pattern in C

```c
int *P = (int*)malloc(5 * sizeof(int));

// use P here

free(P);
```

This is the correct pattern.

---

12. Memory Leak

Meaning of Memory Leak

A memory leak happens when heap memory is created but not released.

Bad example:

```c
int *P = (int*)malloc(5 * sizeof(int));

P[0] = 10;
P[1] = 20;

// forgot free(P)
```

The program requested memory, used it, but did not return it.

That unused memory stays occupied while the program is running.

Simple Example

It is like borrowing a calculator from someone and never returning it.

At first, it may not look serious.

But if you keep borrowing calculators and never return them, eventually nobody has a calculator left.

Memory leaks work in a similar way.

One small leak may not immediately crash a program.

Many leaks can make the program slow or unstable.

---

13. Creating a Heap Array in C++ Using new[]

In C++, dynamic arrays are commonly created using `new[]`.

Example:

```
int *P = new int[5];
```

This creates space for 5 integers in heap memory.

Full C++ Example

```
#include <iostream>
using namespace std;

int main() {
    int *P = new int[5];

    P[0] = 3;
    P[1] = 5;
    P[2] = 7;
    P[3] = 9;
    P[4] = 11;

    for (int i = 0; i < 5; i++) {
        cout << P[i] << " ";
    }

    delete[] P;

    return 0;
}
```

Output:

```
3 5 7 9 11
```

---

14. Releasing Heap Arrays in C++ Using delete[]

If you create a heap array using `new[]`, you must release it using `delete[]`.

Correct:

```
int *P = new int[5];

// use P here

delete[] P;
```

Wrong:

```
int *P = new int[5];

delete P;
```

The wrong version uses `delete` instead of `delete[]`.

Important Rule

For one value:

```
int *P = new int;
delete P;
```

For an array:

```
int *P = new int[5];
delete[] P;
```

Simple memory aid:

```
new      → delete
new[]    → delete[]
```

---

15. Stack Array vs Heap Array

Feature	Stack Array	Heap Array
Example in C	`int A[5];`	`int *P = (int*)malloc(5 * sizeof(int));`
Example in C++	`int A[5];`	`int *P = new int[5];`
Memory area	Stack	Heap
Size	Fixed	Can be decided at runtime
Needs pointer	No	Yes
Created manually	No	Yes
Released manually	No	Yes
C release method	Not needed	`free(P)`
C++ release method	Not needed	`delete[] P`
Main risk	Size limitation	Memory leak
Good for	Small known-size arrays	Arrays whose size is known during runtime

---

16. When to Use Stack Arrays

Use a stack array when:

- The size is small.
- The size is known before running the program.
- You do not need the array after the function ends.

Example:

```
int marks[5] = {80, 75, 90, 65, 88};
```

This is simple and clean.

---

17. When to Use Heap Arrays

Use a heap array when:

- The size is decided while the program is running.

- The array may be large.
- You need more control over memory.

Example:

```
Enter number of students: 50
```

The program cannot know this number before running.

So we can create memory dynamically.

---

18. C Program: User Decides the Array Size

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;

    printf("Enter number of marks: ");
    scanf("%d", &n);

    int *marks = (int*)malloc(n * sizeof(int));

    if (marks == NULL) {
        printf("Memory allocation failed.");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("Enter mark %d: ", i + 1);
        scanf("%d", &marks[i]);
    }

    printf("Marks are:\n");

    for (int i = 0; i < n; i++) {
        printf("%d ", marks[i]);
    }

    free(marks);
```

```
    return 0;
}
```

Explanation

This line asks the user for the size:

```
scanf("%d", &n);
```

This line creates the heap array:

```
int *marks = (int*)malloc(n * sizeof(int));
```

If the user enters 5, memory for 5 integers is created.

If the user enters 100, memory for 100 integers is created.

This is why heap arrays are flexible.

---

19. Printing Heap Array Addresses

A heap array is also stored contiguously if it is created as one block.

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *A = (int*)malloc(5 * sizeof(int));

    if (A == NULL) {
        printf("Memory allocation failed.");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        A[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
```

```
        printf("A[%d] = %d, Address = %p\n", i, A[i], (void*)&A[i]);
    }

    free(A);

    return 0;
}
```

Possible output:

```
A[0] = 10, Address = 1000
A[1] = 20, Address = 1004
A[2] = 30, Address = 1008
A[3] = 40, Address = 1012
A[4] = 50, Address = 1016
```

The exact addresses will be different on your computer.

The important part is the pattern.

If `int` takes 4 bytes, the address usually increases by 4.

---

20. Copying One Heap Array to Another

Copying arrays is important because it prepares you for resizing.

A normal array cannot grow directly.

So the usual method is:

```
1. Create a new bigger array.
2. Copy old values into it.
3. Release old memory.
4. Use the new array.
```

Week 2 focuses on the copying part.

C Example: Copying a Heap Array

```
#include <stdio.h>
#include <stdlib.h>
```

```c
int main() {
    int *P = (int*)malloc(5 * sizeof(int));

    if (P == NULL) {
        printf("Memory allocation failed.");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        P[i] = i + 1;
    }

    int *Q = (int*)malloc(5 * sizeof(int));

    if (Q == NULL) {
        printf("Memory allocation failed.");
        free(P);
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        Q[i] = P[i];
    }

    printf("Copied array:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", Q[i]);
    }

    free(P);
    free(Q);

    return 0;
}
```

Output:

```
Copied array:
1 2 3 4 5
```

Memory View

Before copying:

```
P → [1] [2] [3] [4] [5]
Q → [ ] [ ] [ ] [ ] [ ]
```

After copying:

```
P → [1] [2] [3] [4] [5]
Q → [1] [2] [3] [4] [5]
```

Important:

```
P and Q are two different memory blocks.
They contain the same values, but they are not the same array.
```

---

21. Why Arrays Cannot Grow Directly

Arrays need contiguous memory.

That means all elements must be placed side by side.

Example:

```
A → [1] [2] [3] [4] [5]
```

If we want to add more elements, we need more space after the last element.

But the memory after the array may already be used by another variable or another array.

Example:

```
A → [1] [2] [3] [4] [5] [Used by something else]
```

Because of this, the array cannot simply expand forward.

The safe solution is:

```
Create a bigger block somewhere else.
Copy the old values.
```

```
Release the old block.
Point to the new block.
```

This idea becomes very important in Week 3.

---

22. Simple Resizing Preview

This is only a preview because resizing is mainly Week 3.

Example idea:

```
Old array:
P → [1] [2] [3] [4] [5]

New bigger array:
Q → [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

Copy values:

```
Q → [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
```

Then release old memory:

```
free(P)
```

Then make `P` point to the new array:

```
P = Q
```

Now:

```
P → [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
```

This is the basic idea behind manual dynamic array resizing.

---

23. Common Week 2 Mistakes

Mistake 1: Forgetting `#include <stdlib.h>` in C

Wrong:

```c
#include <stdio.h>

int *P = (int*)malloc(5 * sizeof(int));
```

Correct:

```c
#include <stdio.h>
#include <stdlib.h>
```

`malloc` and `free` need `stdlib.h`.

Mistake 2: Forgetting to Check `malloc`

Weak code:

```c
int *P = (int*)malloc(5 * sizeof(int));
P[0] = 10;
```

Better code:

```c
int *P = (int*)malloc(5 * sizeof(int));

if (P == NULL) {
    printf("Memory allocation failed.");
    return 1;
}

P[0] = 10;
```

Mistake 3: Forgetting `free`

Wrong:

```
int *P = (int*)malloc(5 * sizeof(int));

P[0] = 10;
P[1] = 20;

// no free(P)
```

Correct:

```
free(P);
```

Mistake 4: Using `delete` Instead of `delete[]`

Wrong:

```
int *P = new int[5];
delete P;
```

Correct:

```
int *P = new int[5];
delete[] P;
```

Mistake 5: Accessing Outside the Array

Wrong:

```
int *P = (int*)malloc(5 * sizeof(int));
P[5] = 100;
```

Valid indexes are:

```
0, 1, 2, 3, 4
```

`P[5]` is outside the array.

Mistake 6: Using Memory After Freeing It

Wrong:

```
free(P);
printf("%d", P[0]);
```

After `free(P)`, the memory no longer belongs to your program.

Do not use it again.

A safer habit is:

```
free(P);
P = NULL;
```

Then the pointer clearly points to nothing.

---

24. Stack and Heap Memory Diagram

Stack Array

Code:

```
int A[5] = {2, 4, 6, 8, 10};
```

Memory view:

```
Stack memory:
A → [2] [4] [6] [8] [10]
```

The array itself is stored in stack memory.

Heap Array

Code:

```c
int *P = (int*)malloc(5 * sizeof(int));
```

Memory view:

```
Stack memory:
P → stores address

Heap memory:
[ ] [ ] [ ] [ ] [ ]
```

After values are assigned:

```
Stack memory:
P → address of heap block

Heap memory:
[3] [5] [7] [9] [11]
```

Important sentence:

```
The pointer variable is on the stack, but the array values are on the heap.
```

---

25. Full Week 2 Practice Program in C

This program covers dynamic size, heap memory, pointer usage, `malloc`, indexing, address printing, and `free`.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;

    printf("Enter array size: ");
    scanf("%d", &n);

    int *A = (int*)malloc(n * sizeof(int));

    if (A == NULL) {
```

```
        printf("Memory allocation failed.");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("Enter value for A[%d]: ", i);
        scanf("%d", &A[i]);
    }

    printf("\nArray values and addresses:\n");

    for (int i = 0; i < n; i++) {
        printf("A[%d] = %d, Address = %p\n", i, A[i], (void*)&A[i]);
    }

    free(A);
    A = NULL;

    return 0;
}
```

What This Program Teaches

Code Part	Meaning
`int n;`	Stores the user-given array size
`malloc(n * sizeof(int))`	Creates heap memory for `n` integers
`if (A == NULL)`	Checks whether memory allocation failed
`A[i]`	Accesses each heap array element
`&A[i]`	Gets the memory address of each element
`free(A)`	Releases heap memory
`A = NULL`	Prevents accidental reuse of old pointer

---

26. Full Week 2 Practice Program in C++

```
#include <iostream>
using namespace std;

int main() {
    int n;
```

```
    cout << "Enter array size: ";
    cin >> n;

    int *A = new int[n];

    for (int i = 0; i < n; i++) {
        cout << "Enter value for A[" << i << "]: ";
        cin >> A[i];
    }

    cout << "\nArray values:\n";

    for (int i = 0; i < n; i++) {
        cout << "A[" << i << "] = " << A[i] << endl;
    }

    delete[] A;
    A = nullptr;

    return 0;
}
```

What This Program Teaches

Code Part	Meaning
`new int[n]`	Creates a heap array of size `n`
`A[i]`	Accesses each element
`delete[] A`	Releases the heap array
`A = nullptr`	Makes the pointer point to nothing

---

27. Key Terms

Term	Simple Meaning
Stack	Automatic memory used for local variables
Heap	Memory used for dynamic allocation
Stack array	Fixed-size array usually stored in stack memory
Heap array	Array created dynamically in heap memory

Term	Simple Meaning
Pointer	Variable that stores a memory address
`malloc`	C function used to allocate heap memory
`free`	C function used to release heap memory
`new[]`	C++ operator used to create a heap array
`delete[]`	C++ operator used to release a heap array
Memory leak	Heap memory that was allocated but not released
`NULL`	Means the pointer points to nothing in C
`nullptr`	Modern C++ way to say the pointer points to nothing
Dynamic memory	Memory created while the program is running
Contiguous memory	Memory placed side by side

---

28. Week 2 Summary

By the end of Week 2, you should understand that:

- Stack arrays are normal fixed-size arrays.
- Heap arrays are created dynamically while the program is running.
- Stack memory is automatic.
- Heap memory must be managed manually.
- A pointer stores a memory address.
- A heap array needs a pointer because the program must remember where the heap block starts.
- In C, `malloc` creates heap memory.
- In C, `free` releases heap memory.
- In C++, `new[]` creates a heap array.
- In C++, `delete[]` releases a heap array.
- Forgetting to release heap memory causes a memory leak.
- Heap arrays are useful when the array size is known only during runtime.
- Copying arrays is important before learning resizing.
- Accessing outside the array is dangerous.
- Using memory after freeing it is dangerous.

---

29. Week 2 Checkpoint Questions

Try answering these without looking at the notes.

1. What is stack memory?
2. What is heap memory?
3. What is a stack array?
4. What is a heap array?
5. Why does a heap array need a pointer?
6. What does a pointer store?
7. What does `malloc` do?
8. Why do we use `sizeof(int)` with `malloc`?
9. What does `free` do?
10. What is a memory leak?
11. What does `new[]` do in C++?
12. What does `delete[]` do in C++?
13. Why is `delete[]` different from `delete`?
14. When should you use a stack array?
15. When should you use a heap array?
16. What happens if you access `P[5]` in a 5-element array?
17. Why should you check whether `malloc` returned `NULL`?
18. Why should you not use a pointer after `free(P)`?
19. What is the difference between `int A[5]` and `int *P = malloc(...)`?
20. Why is copying arrays important before learning resizing?

---

30. Week 2 Practice Tasks

Task 1: Stack Array

Write a C program that:

1. Creates a stack array of 5 integers.
2. Assigns values to it.
3. Prints all values using a loop.
4. Prints the address of each element.

Task 2: Heap Array in C

Write a C program that:

1. Asks the user for the array size.
2. Creates a heap array using `malloc`.
3. Checks whether memory allocation succeeded.

4. Accepts values from the user.
5. Prints all values.
6. Prints all addresses.
7. Releases memory using `free`.

Task 3: Heap Array in C++

Write a C++ program that:

1. Asks the user for the array size.
2. Creates a heap array using `new[]`.
3. Accepts values from the user.
4. Prints all values.
5. Releases memory using `delete[]`.

Task 4: Copy Heap Array

Write a C program that:

1. Creates a heap array `P` with 5 values.
2. Creates another heap array `Q` with the same size.
3. Copies all values from `P` to `Q`.
4. Prints values of `Q`.
5. Releases both arrays.

---

31. Simple Final Memory Aid

```
Stack = automatic memory
Heap = manual memory
Stack array = fixed-size array
Heap array = dynamic array
Pointer = stores address
malloc = creates heap memory in C
free = releases heap memory in C
new[] = creates heap array in C++
delete[] = releases heap array in C++
Memory leak = forgot to release heap memory
sizeof(int) = size of one integer
NULL = allocation failed or pointer points to nothing
```

---

32. Final Beginner Explanation

A stack array is like a fixed lunch box with 5 spaces. Once you buy it, you cannot magically turn it into a 10-space lunch box.

A heap array is like asking for a box while the program is running. You can decide the size later. But because you asked for it manually, you must also return it manually.

In C:

```
Ask for memory  → malloc
Return memory   → free
```

In C++:

```
Ask for array memory  → new[]
Return array memory   → delete[]
```

The pointer is the label that tells the program where the box is stored.

So the most important Week 2 idea is:

```
Heap arrays are controlled through pointers, and you must release heap memory
after using it.
```